Unit 2: Python Tools for Data Handling: GUI, APIs, Web Data and Databases

Build interactive applications • Fetch data from the web • Scrape web content • Work with databases • Handle and process data

Python powers data!

1. Python GUI Programming Basics

- Introduction to GUI development options
- Building simple interfaces with Tkinter widgets



Tkinter widgets (create the interface)

- Label
- Entry
- Button

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("Simple GUI")
label = ttk.Label(root, text="Enter your name:")
entry = ttk.Entry(root)
button = ttk.Button(root, text="Submit")
button pack()
root.mainloop()
```

GUI code (using Tkinter)

2. Working in Jupyter Notebook/IPython

- Interactive data science environment
- Run Python code, visualize results, explore data

jupyter

Jupyter Notebook

Run code in a cell

```
In [1]:
import pandas as pd
df = pd.read_csv("data.csv")
df.head()
```

Out[1]:

	Name	Age	City
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	22	Bangalore

View output (data, plots, etc.)

IP IPython

3. Internet Programming – APIs

REST API

- Network Scripting basics
- Working with REST APIs using the requests library (JSON responses, authentication basics)

HTTP request (using requests)

```
import requests
url = "https://jsonplaceholder.typicode.com/posts/1"
response = requests.get(url)
data = response.json()
print(data)  # JSON response
```

JSON Response

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt",
  "body": "..."
}
```

Authentication (basic example)

```
headers = {"Authorication": "Bearer <token>"}
response = requests.get(url, headers=headers)
```

Authentication (with headers)

4. Web Scraping Basics

- Extract data from websites using BeautifulSoup



Parse HTML and extract data

BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
url = "https://example.com/products"
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')
products = soup.find_all('div', class_='product')
for product in products:
    name = product.find('h2').text
    price = product.find('span', class_='price').text
    print(name, price)
```

Scraping code (BeautifulSoup)

Extracted Data

Product Name	$10
Product 2	$20
Product 3	$30
...	...

5. Databases and Persistence

- SQL fundamentals
- Connect Python to databases using sqlite3/SQLAlchemy
- Working with data structures, text and language for data handling

Database connection (using sqlite3 / SQLAlchemy)

SQLite SQLAlchemy

SQL operations (create, insert, select)

```
import sqlite3
conn = sqlite3.connect('data.db')
cursor = conn.cursor()
cursor.execute("CREATE TABLE users (id INTEGER, name TEXT)")
cursor.execute("INSERT INTO users VALUES (1, 'Alice')")
conn.commit()
cursor.execute("SELECT * FROM users")
print(cursor.fetchall())
conn.close()
```

Users Table

id	name
1	Alice
2	Bob
3	Charlie

6. Working with Data Structures, Text and Language

- Use lists, dictionaries, strings and basic NLP tools for data handling

Lists (store data)

```
data = [1, 2, 3, 4]
print(data[0])
```

Accessing element

Dictionaries (key-value pairs)

```
info = {'name': 'Alice',
        'age': 25}
print(info['name'])
```

Accessing value

Text Processing

```
text = "Python is great!"
words = text.split()
print(words)
```

Tokenizing text

Basic NLP (using NLTK)

```
import nltk
nltk.download('punkt')
tokens = nltk.word_tokenize(text)
```

Language processing

Text Lists Dictionaries NLP

Get Data → Process → Analyze → Build Smarter Apps

Python + GUI + APIs + Web Scraping + Databases + Data Handling = Real-World Solutions

Python Tools for Data Handling: GUI, APIs, Web Data and Databases

1. Python GUI Programming Basics

What is Python GUI Programming?

GUI means Graphical User Interface. It helps us build windows, buttons and menus using Python.

We can use the tkinter library, which comes with Python, to create GUI applicactions.

Example Code

```
import tkinter as tk

def greet():
    label.config(text="Hello, Python!")

app = tk.Tk()
app.title("My First GUI")

label = tk.Label(app, text="Welcome!")
label.pack(pady=10)

btn = tk.Button(app, text="Click Me", command=greet)
btn.pack()
app.mainloop()
```

2. GUI Elements in Tkinter

What are the main GUI elements?

We can use widgets like Label, Button, Entry, Text, Checkbutton, and more.

Common Widgets



3. Working with APIs and Web Data

Python also helps us fetch data from the web using APIs. We can use the requests library.

Yes! For example, we can get live data from a weather API and show it in our application.

Fetching Web Data

```
import requests

response = requests.get(
    "https://api.openweatheramp.org/data/2.5/weather?q=London&appid=API_KEY"
)

data = response.json()
temp = data["main"]["temp"]
print(f"Temperature: {temp}°C")
```

4. Working with Databases

To store and manage data, we can use databases. Python supports SQLite, which is easy to use.

Using the sqlite3 module, we can create tables, insert data and fetch records.

Database Example

```
import sqlite3
conn = sqlite3.connect("data.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE users(name TEXT)")
cursor.execute("INSERT INTO users VALUES (?)", ("Alice",))
conn.commit()
conn.close()
```

5. Summary

So, Python GUI, APIs, and Databases together help us build powerful and interactive applications.

Exactly! With these tools, we can handle data, fetch information from the web, and create user-friendly apps.

Key Takeaways

- Use tkinter for GUI programming
- Fetch data with APIs (requests)
- Work with databases (sqlite3)
- Build complete applications!

Python makes data handling simple and powerful!

Introduction to GUI Development Options

Explore different ways to build Graphical User Interfaces in Python!

Hi! I want to build a desktop application in Python. What are my options for GUI development?

Great question! Python offers several GUI frameworks. Let's explore the popular options.

1. Tkinter

Tkinter is Python's built-in GUI library. It's simple and easy to use.



- Built-in with Python
- Simple and lightweight
- Great for beginners

2. PyQt

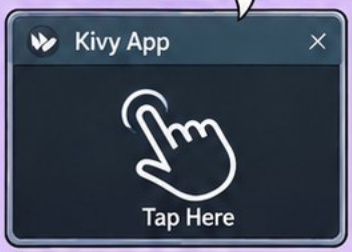
PyQt is a powerful framework that provides a modern look and more features.



- Rich features
- Modern UI
- Suitable for complex apps

3. Kivy

Kivy is great for building cross-platform apps, including touch-enabled apps.



- Cross-platform
- Touch support
- Flexible design

4. wxPython

wxPython lets you build native-looking apps using the wxWidgets toolkit.



- Native look & feel
- Multiple platforms
- Feature rich

So, which one should I choose?

It depends on your project!

- Tkinter – for simplicity
- PyQt – for advanced features
- Kivy – for cross-platform & touch apps
- wxPython – for native look

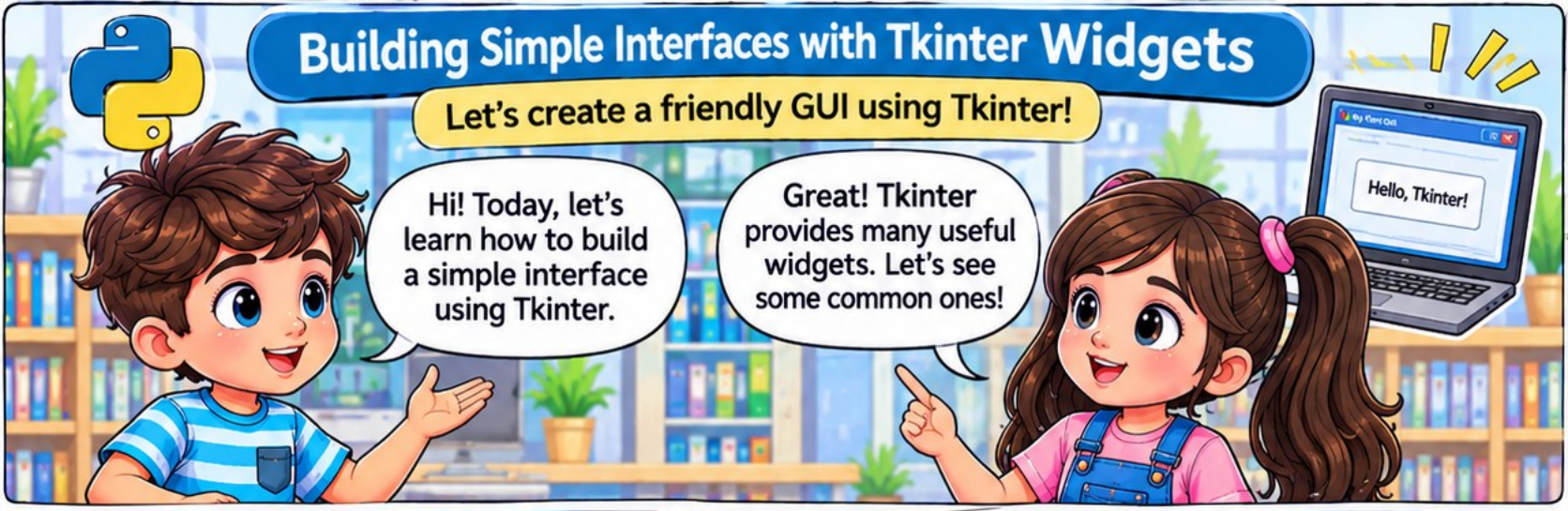
Summary & Key Takeaways

- Python offers multiple GUI development options.
- Tkinter is the simplest choice.
- PyQt and wxPython provide advanced features.
- Kivy is ideal for cross-platform apps.
- Choose the right tool based on your needs!

Start building amazing GUI apps with Python!

Building Simple Interfaces with Tkinter Widgets

Let's create a friendly GUI using Tkinter!



1. Common Tkinter Widgets

Label

Hello!

Button

Click Me

Entry

Type here...

Text

This is a text box.

Checkbutton

Agree

Radiobutton

Option 1

2. Example: A Simple Tkinter App

```
import tkinter as tk

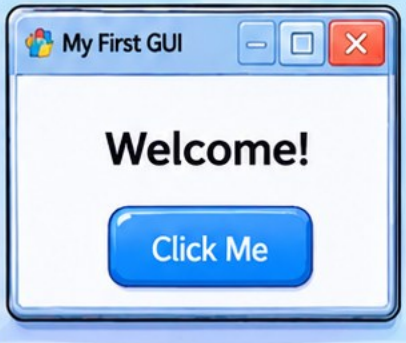
def show_message():
    label.config(text="Hello,
      Tkinter!")

app = tk.Tk()
app.title("My First GUI")

label = tk.Label(app, text="Welcome!",
 font=("Arial", 14))
label.pack(pady=10)

button = tk.Button(app, text="Click Me",
        command=show_message)
button.pack(pady=10)

app.mainloop()
```



3. How It Works?



4. Summary & Key Takeaways

- Tkinter helps us build GUI applications in Python.
- Common widgets: Label, Button, Entry, Text, Checkbutton, Radiobutton.
- Use .pack(), .grid() or .place() to arrange widgets.
- Add interactivity with functions.
- Start your GUI with app.mainloop().

Start creating simple and useful apps with Tkinter!

Working in Jupyter Notebook/IPython as the Data Science Environment

Explore. Analyse. Visualize. Share.

Hi! What is Jupyter Notebook? And how is it different from Python IDLE?

Jupyter Notebook is an interactive environment that lets us write code, run it, and see results – all in one place!

What is IPython?

IPython is an enhanced Python shell that powers Jupyter Notebook, offering:

- Rich output
- Tab completion
- Interactive help

2. Getting Started

1. Install Jupyter Notebook (pip install notebook).
2. Launch it with (jupyter notebook).
3. Open a new notebook and start coding!



3. Key Features

- Write & Run Code in cells
- Visualize Data (with Matplotlib, Seaborn, etc.)
- Explore Data (with Pandas)
- Add Notes & Markdown
- Save & Share Your Work

4. Example: Data Analysis Workflow

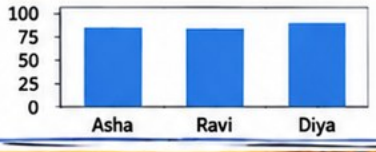
```
In [1]: import pandas as pd
        import matplotlib.pyplot as plt

In [2]: data = pd.read_csv('data.csv')
        data.head()
```

Out[2]:

Name	Age	Score
Asha	12	90
Ravi	13	85
Diya	14	92

```
In [3]: data['Score'].plot(kind='bar')
        plt.show()
```



5. Useful Magic Commands

%matplotlib inline	Show plots inside notebook
%timeit	Measure execution time
%load	Load code from a file
%whos	View variables in the notebook
?function	Get help about a function

These commands make analysis faster and easier!

6. Best Practices

- Use Markdown cells to add explanations.
- Organize code in clear cells.
- Use libraries like Pandas, NumPy, Matplotlib.
- Save your notebook regularly.
- Keep your code modular and reusable.



7. Summary & Key Takeaways

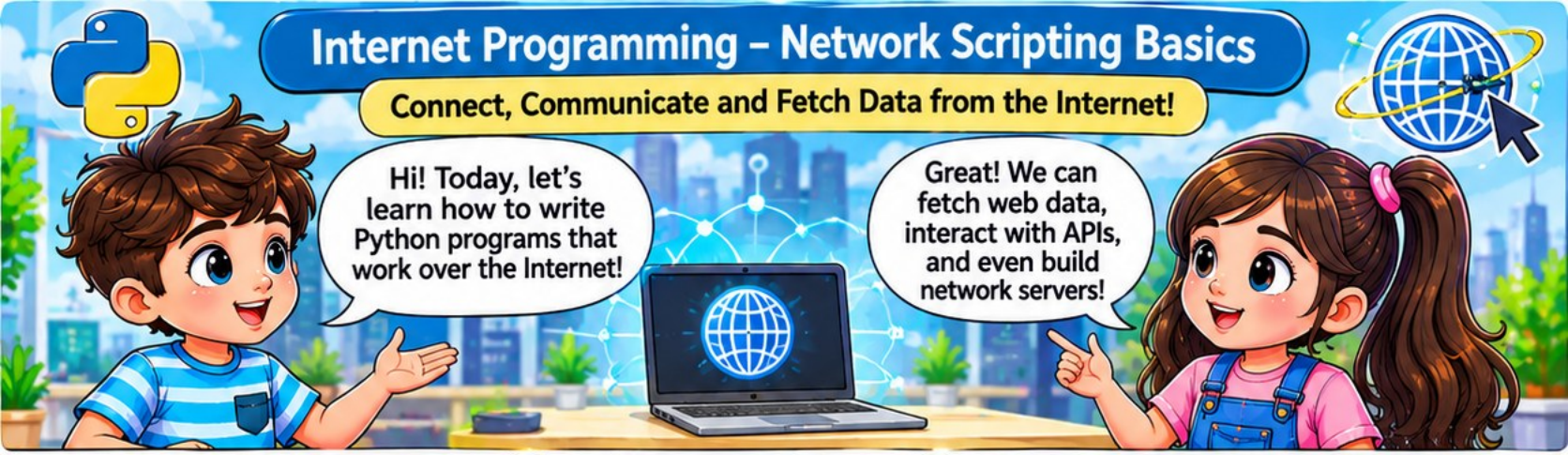
Jupyter Notebook makes Python powerful for Data Science – with an interactive, visual and user-friendly workspace!

- Jupyter Notebook = Interactive Python IDE
- Use cells to write & run code
- Visualize data easily
- Explore data with Pandas
- Share your work

Start your Data Science journey with Jupyter Notebook!

Internet Programming – Network Scripting Basics

Connect, Communicate and Fetch Data from the Internet!



1. What is Network Scripting?

Network scripting means writing programs to send and receive data over a network (like the Internet).



Common Protocols:

- **HTTP/HTTPS** – Web data
- **FTP** – File transfer
- **TCP/UDP** – Communication

2. Basic Tools in Python

Python provides powerful libraries for network programming:

- **requests** – for HTTP requests
- **socket** – for low-level networking
- **urllib** – for URL handling

3. Fetching Web Data with requests

Use the **requests** library to get data from a website.

```
import requests

response = requests.get(
   "https://api.github.com")

print(response.status_code)
print(response.json())
```

Key Points:

- Check status code (e.g., 200)
- Use .json() to get data
- Handle errors

4. Working with Sockets

Sockets are used for real-time communication (e.g., chat apps).




```
import socket

s = socket.socket()
s.connect(('example.com', 80))
s.send(b'Hello Server!')
print(s.recv(1024))
```

5. APIs and Web Services

APIs let you use services like weather info, maps, or social media data.




```
import requests

api_key = "YOUR_KEY"
url = f"https://api.openweathermap.org/
    data/2.5/weather?q=London&appid={api_key}"

response = requests.get(url)
data = response.json()
print(data['weather'][0]['description'])
```

6. Important Concepts

HTTP Methods
GET – read data
POST – send data

Handling Errors
Use try-except blocks.

Timeouts
Prevent hanging.

Working with JSON
Common data format in APIs.

Security
Use HTTPS and be careful with input.



7. Best Practices

- Use libraries like requests.
- Handle errors gracefully.
- Keep your code modular.
- Use proper timeouts.
- Follow security best practices (e.g., verify SSL).



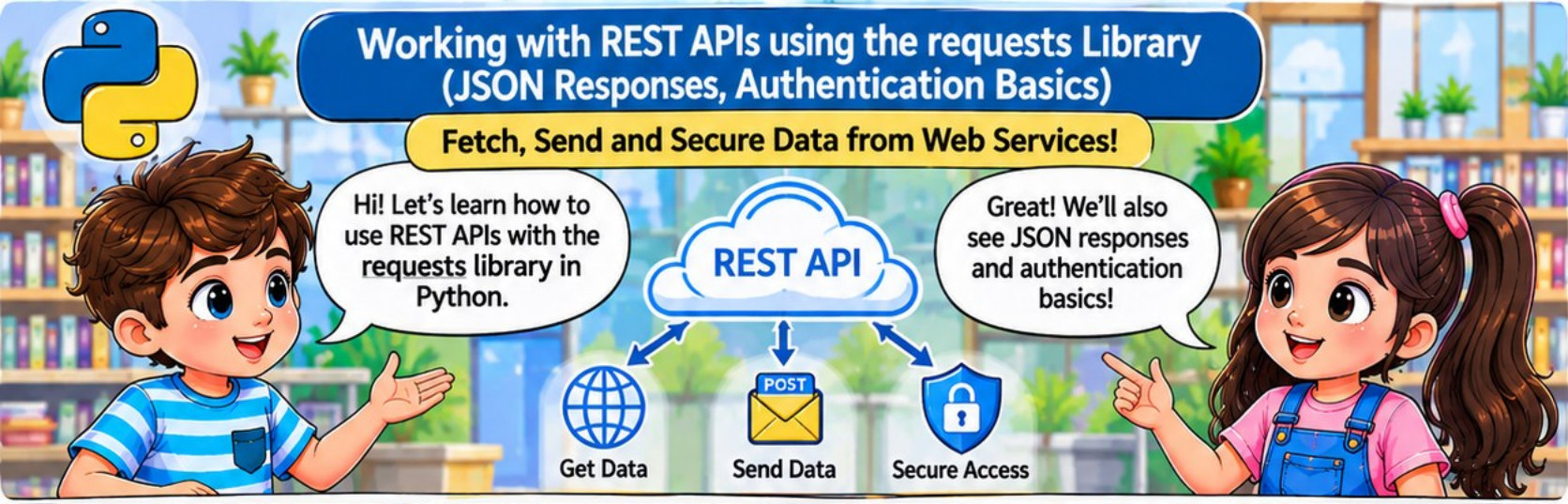
Summary & Key Takeaways

- Network scripting lets you work with the Internet using Python.
- Use requests for web data and socket for real-time communication.
- Learn APIs to access powerful online services.
- Handle errors and follow best practices.
- Now you can explore more – like web scraping, chat applications and data analysis!



Working with REST APIs using the requests Library (JSON Responses, Authentication Basics)

Fetch, Send and Secure Data from Web Services!



1. What is a REST API?

A REST API lets you access web services over HTTP, using simple requests.



Common Methods:

- GET – Read data
- POST – Create data
- PUT – Update data
- DELETE – Remove data

2. Install and Import requests

Install the library (if not installed):

```
pip install requests
```



Import it in your Python code:

```
import requests
```

3. Making a Simple API Request

Get data from a public API (e.g., JSONPlaceholder):

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts"
)

print(response.status_code)
print(response.json())
```

Output: (JSON Data)

```
[{'userId': 1,
  'id': 1,
  'title': 'sunt
   aut facere',
  'body': '...'}]
}
```

4. Working with JSON Data

Use .json() to parse JSON responses:

```
data = response.json()
for post in data[:2]:
    print[post["title"])
```



JSON is a lightweight data format, easy to use!

5. Authentication Basics

Many APIs require authentication. Common methods:

- API Key – Simple key
- Bearer Token – OAuth 2.0
- Basic Auth – Username & Password

Example with API Key:

```
headers = {"Authorication": "Bearer YOUR_API_KEY"}
response = requests.get(url, headers=headers)
```

6. Handling Errors & More Options

Use try-except to handle errors and options like:

```
try:
    response = requests.get(url, timeout=5)
    response.raise_for_status() # Check errors
    print(response.json())
except requests.exceptions.RequestException as e:
    print("Error:", e)
```

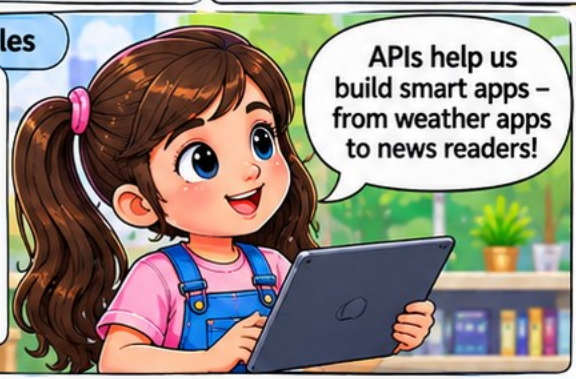
Useful Options:

- params – Query parameters
- headers – Set headers
- timeout – Set time limit
- verify – Handle SSL

7. Real-World Examples

Explore popular APIs:

- OpenWeatherMap (Weather Data)
- GitHub (Repository Data)
- NewsAPI (News Articles)



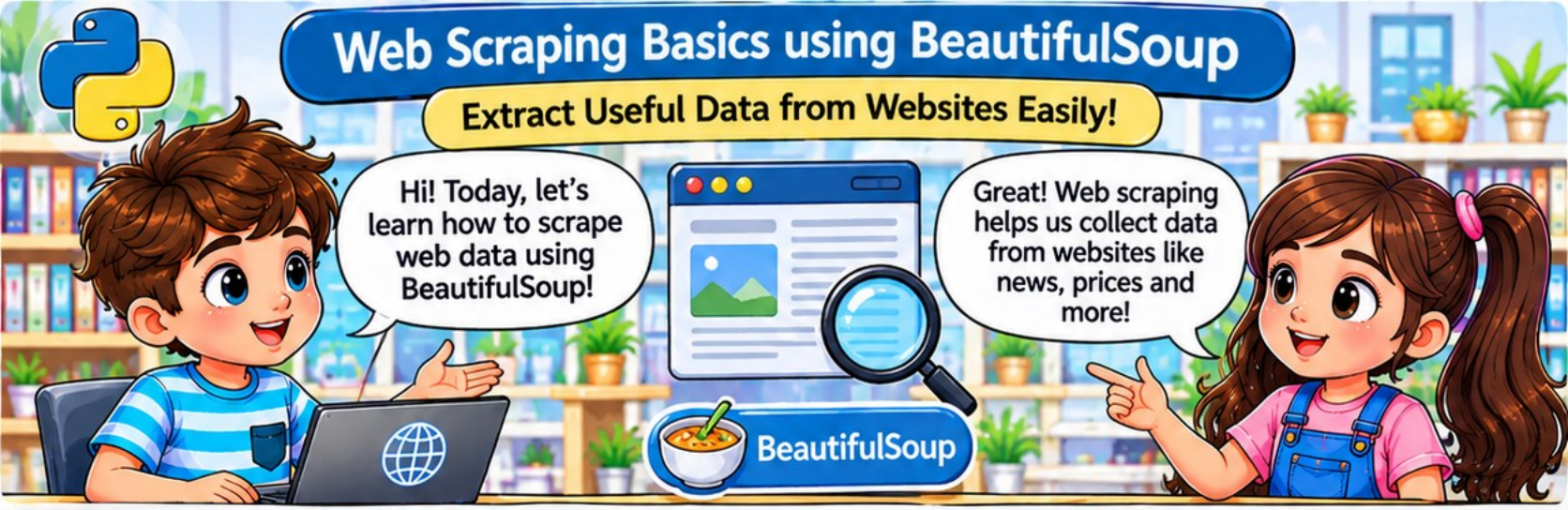
8. Summary & Key Takeaways

- Use requests to call REST APIs.
- Get and parse JSON data.
- Use authentication (API keys/tokens).
- Handle errors and set timeouts.
- Explore real-world APIs for exciting projects!



Web Scraping Basics using BeautifulSoup

Extract Useful Data from Websites Easily!



1. What is Web Scraping?

Web scraping is the process of extracting data from websites automatically.



Common Uses:

- Get latest news
- Track product prices
- Collect research data
- Build datasets

2. Install Required Libraries

Use pip to install BeautifulSoup and requests.

```
pip install beautifulsoup4
pip install requests
```

Import them:

```
import requests
from bs4 import BeautifulSoup
```

3. Fetch a Web Page

Use requests to get the page content.

```
url = "https://example.com"
response = requests.get(url)
html = response.text
```



4. Parse the HTML

Use BeautifulSoup to parse the HTML and find data.

```
soup = BeautifulSoup(html, 'html.parser')
title = soup.find('h1').text
print(title)
```



You can use find(), find_all() and CSS selectors to locate data.

5. Extract Data

Get the data you need — text, links, or attributes.

```
links = soup.find_all('a')
for link in links:
    print(link.get('href'))
```

Useful Selectors:

- Tag: soup.find('div')
- Class: soup.find_all('div', class_='news')
- ID: soup.find(id='main')

6. Handle Real-World Challenges

Web scraping may face issues like:

- Dynamic Content (use Selenium)
- Blocked Requests (use headers & delays)
- Missing Data (use checks)

Helpful Tips:

- Set a User-Agent header
- Add delays (time.sleep)
- Handle exceptions
- Respect robots.txt

7. Summary & Key Takeaways

- Web scraping extracts data from websites using Python.
- Use requests to fetch pages and BeautifulSoup to parse HTML.
- Find data with find(), find_all() and CSS selectors.
- Handle errors and follow website rules.
- Use scraping responsibly!

Start scraping today and turn the web into your data source!

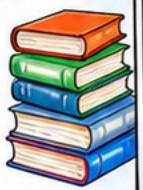
Databases and Persistence – SQL Fundamentals

Store, Manage and Query Data with SQL!



1. What is a Database?

A database is an organized collection of data, like a library for information.



Popular Databases:

- SQLite (lightweight)
- MySQL
- PostgreSQL
- Oracle

2. What is SQL?

SQL (Structured Query Language) is used to:



- Create tables
- Query data
- Insert, update or delete data
- Manage data efficiently

SQL is the standard language for relational databases.

3. Basic SQL Commands

CREATE TABLE	Define a new table
INSERT INTO	Add data
SELECT	Retrieve data
UPDATE	Modify data
DELETE	Remove data

Example: Create and Query

```
CREATE TABLE students (
   id INTEGER PRIMARY KEY,
   name TEXT,
   age INTEGER
);

SELECT * FROM students;
```

id	name	age
1	Alice	14
2	Bob	15

4. Working with Python

Python makes it easy to work with databases using the sqlite3 module.

```
import sqlite3
conn = sqlite3.connect('school.db')
cursor = conn.cursor()
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
   print(row)
```

Don't forget to close the connection!

5. Key Concepts

Tables
Store data in rows and columns.

Primary Key
Uniquely identifies each record.

Relationships
Link tables using foreign keys.

Transactions
Ensure data consistency (commit or rollback).

6. Advanced SQL Features

- WHERE clause – Filter data
- ORDER BY – Sort results
- JOIN – Combine tables
- GROUP BY – Aggregate data
- Aggregate Functions – COUNT(), SUM(), AVG(), etc.
- Indexes – Improve query performance

Example: Join Tables

```
SELECT s.name, c.course
FROM students s
JOIN courses c
ON s.course_id = c.id;
```

name	course
Alice	Python
Bob	Data Science

7. Best Practices



- Use meaningful table names.
- Normalize data to avoid redundancy.
- Use parameterized queries to prevent SQL injection.
- Backup your database regularly.
- Optimize queries for large data.

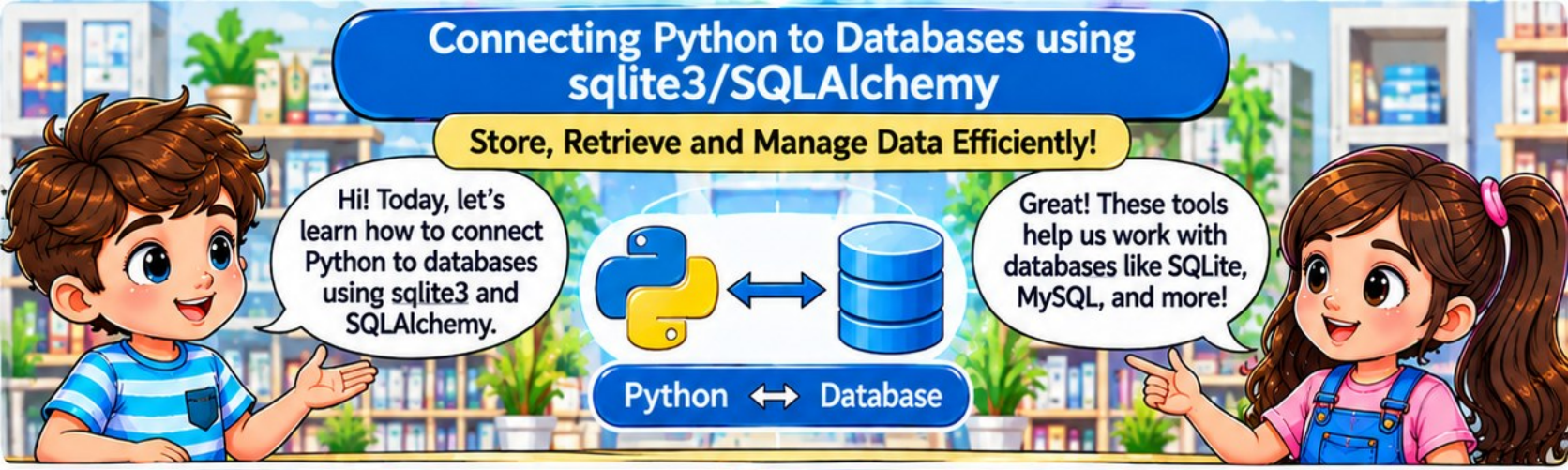
8. Summary & Key Takeaways

- A database helps store data permanently.
- SQL is the language to work with databases.
- You can create, read, update and delete data.
- Python's sqlite3 module makes it easy to use SQL.
- Follow best practices for secure and efficient data handling.



Connecting Python to Databases using sqlite3/SQLAlchemy

Store, Retrieve and Manage Data Efficiently!



1. What is sqlite3?

sqlite3 is Python's built-in library for working with SQLite, a lightweight database.

SQLite

Key Features:

- No separate server
- Simple and easy to use
- Perfect for small projects

2. Using sqlite3

Connect, create tables, and perform database operations.

```
import sqlite3

conn = sqlite3.connect('data.db')
cursor = conn.cursor()

cursor.execute('''
 CREATE TABLE students (
     id INTEGER PRIMARY KEY,
     name TEXT,
     age INTEGER
)''')
conn.commit()
```

3. SQLAlchemy – The ORM

SQLAlchemy is a powerful ORM that simplifies database operations with Python objects.

SQLAlchemy

Advantages:

- Works with multiple databases
- Provides high-level abstraction
- Supports advanced features

4. Connecting with SQLAlchemy

```
from sqlalchemy import create_engine,
text

engine = create_engine(
     "sqlite://data.db")

with engine.connect() as conn:
   result = conn.execute(
     text("SELECT * FROM students")
     )
  for row in result:
     print(row)
```

5. Working with Data

Insert, query and update data easily.

```
from sqlalchemy.orm import Session
from sqlalchemy. import update

with Session(engine) as session:
   session.add(Student(
       name='Alice', age=14))
   session.commit()

   result = session.query(Student).
     filter(Student.age > 12).all()
```

Students Table		
id	name	age
1	Alice	14
2	Bob	15

6. Important Concepts

- **Engine** – Connects to DB
- **Session** – Manages transactions
- **Transactions** – Commit or Rollback changes
- **ORM** – Map tables to Python classes
- **Migrations** – Use Alembic for schema changes

7. Best Practices

- Use context managers (with) for connections
- Use parameterized queries (to prevent SQL injection)
- Handle exceptions
- Optimize queries with indexes
- Backup your database

SQLAlchemy is great for larger projects, while sqlite3 is perfect for simple tasks!

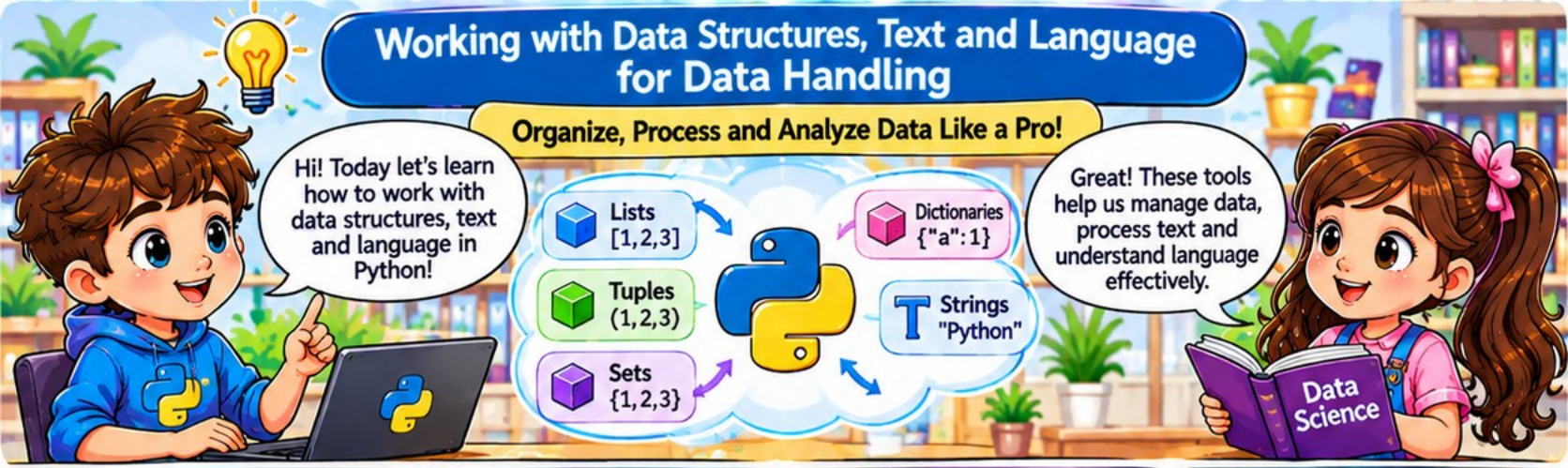
8. Summary & Key Takeaways

- Use sqlite3 for SQLite databases.
- Use SQLAlchemy for a powerful ORM approach.
- Perform CRUD operations (Create, Read, Update, Delete).
- Follow best practices for security and performance.
- Build reliable, data-driven Python applications!



Working with Data Structures, Text and Language for Data Handling

Organize, Process and Analyze Data Like a Pro!



1. Python Data Structures

Data structures help us organize data efficiently.

Lists	Tuples	Sets	Dictionaries
(ordered, mutable)	(ordered, immutable)	(unique elements)	(key-value pairs)

```
fruits = ["apple", "banana", "cherry"]
info = {"name": "Alice", "age": 14}
```

2. Working with Text

Strings are used to handle text data like names, messages, etc.

String Methods:

- upper(), lower()
- strip(), split()
- replace(), find()

```
text = "  Hello Python!  "
print(text.strip().upper())
# Output: HELLO PYTHON!
```

3. Language Features

Python offers powerful features for data handling.

- List Comprehensions:
  squares = [x*x for x in nums]
- Lambda Functions:
  add = lambda a, b: a + b
- Built-in Functions:
  len(), sum(), map(), filter()

Write concise and efficient code!

4. Working with Data

Combine data structures for analysis and processing.

Name	Age	City
Alice	14	Delhi
Bob	15	Mumbai
Charlie	16	Chennai

Use data structures to filter, sort and analyze data.

5. Text Processing Techniques

Clean and prepare text for better analysis.

- Tokenization: split text into words
- Removing stop words
- Stemming & Lemmatization
- Regular Expressions (for pattern matching)

data
science
is
fun

```
import re
words = re.findall(r'\w+', text)
```

6. Handling Language Data

Work with multiple languages and understand text.

- Language Detection:
  langdetect.detect(text)
- Translation:
  from googletrans import Translator
- Sentiment Analysis:
  from textblob import TextBlob

Understand sentiment: Positive, Negative or Neutral!

7. Useful Tools & Libraries

Powerful Python libraries for data and language tasks.

- **pandas** – data analysis
- **NLTK** – NLP tasks
- **spaCy** – advanced NLP
- **TextBlob** – simple NLP
- re - regular expressions

These tools make data handling easy!

8. Best Practices & Key Takeaways

- Choose the right data structure.
- Keep text data clean.
- Use libraries for complex tasks.
- Handle errors gracefully.
- Write modular and reusable code.

Key Takeaways:

Organize data, process text, and use Python's powerful tools to unlock insights!

With the right data tools and techniques, you can turn raw data into meaningful insights!

Happy Coding with Python!